

CS 473 (Spring 2025)

Homework 4 (due Mar 6 Thu 10am)

Instructions: As previous homeworks.

Problem 4.1: Suppose we have a Las Vegas randomized algorithm A that runs in μ expected number of steps. As mentioned in class, for any given $t > 1$, we can obtain a new Monte Carlo algorithm A' that runs within $t\mu$ steps by outputting “garbage” if the time limit is exceeded. The error probability is at most $1/t$ by Markov’s inequality.

But we can do better! For example, we can run A for up to 2μ steps, and if it fails, rerun A for another 2μ steps (with an independent sequence of random bits), etc., for $\lfloor t/2 \rfloor$ rounds. This gives a Monte Carlo algorithm with the total number of steps still bounded by $t\mu$, and the error probability reduced to $1/2^{\lfloor t/2 \rfloor} = O(1/2^{t/2}) \leq O(0.708^t)$, which decays exponentially!

In this question, you will analyze further improvements to the base of the exponential:

- (a) (25 pts) Let $X \geq 0$ be a random variable with $\mathbb{E}[X] = \mu$. Given $\alpha_1, \alpha_2, \dots, \alpha_k \geq 0$, let $p_1 = \Pr[X \geq \alpha_1\mu]$, $p_2 = \Pr[X \geq (\alpha_1 + \alpha_2)\mu]$, ..., $p_k = \Pr[X \geq (\alpha_1 + \dots + \alpha_k)\mu]$. Prove the following inequality:

$$\alpha_1 p_1 + \alpha_2 p_2 + \dots + \alpha_k p_k \leq 1.$$

[Hint: modify the proof of Markov’s inequality and use telescoping sum...]

- (b) (25 pts) Now, consider a strategy with three iterations where we run A for $\alpha_1\mu$ steps in the first iteration, $(\alpha_1 + \alpha_2)\mu$ steps in the second iteration, and $(\alpha_1 + \alpha_2 + \alpha_3)\mu$ steps in the third iteration.

With this strategy, we obtain a Monte Carlo algorithm with the total number of steps upper-bounded by $(3\alpha_1 + 2\alpha_2 + \alpha_3)\mu$. Bound the error probability purely as a function of α_1 , α_2 , and α_3 .

[Hint: you may use the fact that $x + y + z \leq 1$ ($x, y, z \geq 0$) implies $xyz \leq 1/27$. (This follows from the standard inequality that the “geometric mean” is at most the “arithmetic mean”).]

- (c) (25 pts) Using part (b), show how to obtain a Monte Carlo algorithm with the total number of steps bounded by $t\mu$, and the error probability at most $O(0.545^t)$ (thus, improving $O(0.708^t)$).

[Hint: repeat for $\left\lfloor \frac{t}{3\alpha_1 + 2\alpha_2 + \alpha_3} \right\rfloor$ rounds; set $\alpha_1 = c/3$, $\alpha_2 = c/2$, $\alpha_3 = c$ for some choice of c (you might need a calculator)...]

- (d) (25 pts) Generalize part (b) from 3 iterations to k iterations for a larger constant k , and show how to reduce the error probability further to below $O(0.4^t)$.

[Hint: choose some k below 50, and choose $c = 1$...]

Solution:

(a) Assume $\beta_i = (\alpha_1 + \dots + \alpha_i)\mu, \beta_0 = 0$,

$$\begin{aligned}\mathbb{E}[X] &= \int_0^\infty \Pr[X \geq t] dt \\ &= \sum_{i=1}^k \int_{\beta_{i-1}}^{\beta_i} \Pr[X \geq t] dt + \int_{\beta_k}^\infty \Pr[X \geq t] dt\end{aligned}$$

Since $\Pr[X \geq t]$ is non-increase in t ,

$$\begin{aligned}\int_{\beta_{i-1}}^{\beta_i} \Pr[X \geq t] dt &\leq \Pr[X \geq \beta_{i-1}](\beta_i - \beta_{i-1}) \\ &= p_i \alpha_i \mu\end{aligned}$$

Thus,

$$\begin{aligned}\mathbb{E}[X] &\leq \sum_{i=1}^k p_i \alpha_i \mu \\ \sum_{i=1}^k \alpha_i p_i &\leq 1\end{aligned}$$

(b) The probability that the Monte Carlo algorithm fails is $p_1 p_2 p_3$, from (a),

$$\alpha_1 p_1 + \alpha_2 p_2 + \alpha_3 p_3 \leq 1 \quad (\alpha_1 p_1, \alpha_2 p_2, \alpha_3 p_3 \geq 0)$$

Thus,

$$\begin{aligned}\alpha_1 p_1 \alpha_2 p_2 \alpha_3 p_3 &\leq \frac{1}{27} \\ \Pr(\text{error}) = p_1 p_2 p_3 &\leq \frac{1}{27 \alpha_1 \alpha_2 \alpha_3}\end{aligned}$$

(c) Let $\alpha_1 = \frac{c}{3}, \alpha_2 = \frac{c}{2}, \alpha_3 = \frac{c}{1}$, the Monte Carlo algorithm repeats for $\lfloor \frac{t}{3c} \rfloor$ rounds, each round takes $3c\mu$ steps. The total number of steps is bounded by,

$$\left\lfloor \frac{t}{3c} \right\rfloor \cdot 3c\mu \leq t\mu$$

The probability that the algorithm fails is,

$$\Pr(\text{error}) \leq \left(\frac{6}{27c^3}\right)^{\lfloor \frac{t}{3c} \rfloor} = O\left(\left(\frac{6}{27c^3}\right)^{\frac{t}{3c}}\right)$$

Let $c = \frac{\sqrt[3]{6}e}{3} \approx 1.646$, we get,

$$\Pr(\text{error}) \leq O(0.545^t)$$

- (d) Let $\alpha_i = \frac{1}{k+1-i}$ for $i = 1, \dots, k$. The Monte Carlo algorithm repeats for $\lfloor \frac{t}{k} \rfloor$ rounds, each round takes $k\mu$ steps. The total number of steps is bounded by $t\mu$. The probability that each round fails is,

$$\prod_{i=1}^k p_i \leq k^{-k} \prod_{i=1}^k \alpha_i^{-1} = k^{-k} k!$$

The probability that the algorithm fails is,

$$\Pr(\text{error}) \leq (k^{-k} k!)^{\lfloor \frac{t}{k} \rfloor} = O((k^{-k} k!)^{\frac{t}{k}})$$

$\Pr(\text{error})$ is decreasing when $k > 0$, let $k \geq 32$,

$$\Pr(\text{error}) \leq O(0.3997^t) \leq O(0.4^t)$$

Problem 4.2: In a popular form of logic puzzles, you land on an island that has three types of inhabitants: “knights”, who always tell the truth; “knaves”, who always lie; and “spies”, who sometimes lie and sometimes tell the truth.

Suppose there are n inhabitants, where 60% are known to be decent folks, i.e., knights. The remaining 40% are bad, i.e., knaves or spies. You want to know who are the good/bad guys, i.e., determine the types of all n inhabitants. You are allowed to ask only questions of the form, “is person A a knight/knave/spy?”, to another person B. (All the inhabitants know each other.) Obviously, if you can find a person who you know is a knight, the problem can be solved after asking n additional questions.

- (a) (10 pts) Give a (very) efficient Monte Carlo algorithm that finds a knight. State the probability of error. (Hint: this is supposed to be very easy!)
- (b) (90 pts) Give a Las Vegas algorithm that finds a knight by asking $O(n)$ expected number of questions. Analyze the constant factor in the big-Oh and make it smaller than 1.5.
[Hint: use (a). How can you confirm whether a specific person is a knight by asking $O(n)$ questions?]
[Note: there is also a deterministic algorithm that requires $O(n)$ questions, but it is more complicated and has a larger constant factor.]

Solution:

- (a) Monte Carlo algorithm:
Randomly select a person A and accept A as a knight.
The probability of error is 0.4.
- (b) Las Vegas algorithm:
 1. Select a random person A .
 2. Select a group of $0.8n$ inhabitants and ask each one of them whether A is a knight.
 3. A is a knight if more than half of the group agree since there are at most $0.4n$ knaves.
 4. Otherwise, repeat step 1 $\sim$ 3 until we find a knight.
 The expected number of questions is,

$$\begin{aligned}
 \mathbb{E}(\text{questions}) &= \sum_{i=1}^{\infty} i \cdot (0.4)^{i-1} \cdot 0.6 \cdot 0.8n \\
 &= \sum_{i=0}^{\infty} (i+1) \cdot (0.4)^i \cdot 0.48n \\
 &= \left(\frac{0.4}{(1-0.4)^2} + \frac{1}{1-0.4} \right) \cdot 0.48n \\
 &\approx 1.333n < 1.5n
 \end{aligned}$$

Problem 4.3: We are given a set S of strings of total length N over the binary alphabet $\{0, 1\}$. We are also given a number ℓ . We want to find a palindrome p of length ℓ and two distinct strings $s, s' \in S$ such that their concatenation ss' contains p as a substring.

For example, for $S = \{0111, 1111010, 10110, 11011\}$ and $\ell = 7$, the concatenation of 1111010 and 11011 contains the palindrome 1101011.

Describe a randomized Las Vegas algorithm for this problem, by using the Karp–Rabin fingerprint technique (to be covered in Feb 28’s lecture). The expected running time should be $O(N \log N)$ or better.

[Hint: for every string $s \in S$, pre-compute fingerprints of each prefix/suffix of s and the reverse of s . You may find subtraction useful...]

Solution:

1. For every string $s \in S$, pre-compute Karp-Rabin fingerprints of each prefix/suffix x of s and the reverse of s as $F(x)$ and $F(x^R)$.
2. For every string $s \in S$, use a slide window of length ℓ to find whether there is a palindrome within s .

For each suffix x of length i and prefix y of length $\ell - i$, xy might be a palindrome if

$$F(x) \cdot b^{\ell-i} + F(y) = F(y^R) \cdot b^i + F(x^R)$$

where b is the base in Karp-Rabin fingerprints.

3. Sort all prefix y according to $F(y^R) \cdot b^i - F(y)$.

For each suffix x , compute $F(x) \cdot b^{\ell-i} - F(x^R)$ and check if there is a corresponding y using binary search.

4. If there is a possible match, verify if it is a palindrome.

If not, find palindrome by brute force.

5. Repeat step 3 and 4 until the palindrome is found.

Runtime Analysis:

S has at most $\frac{2N}{\ell}$ strings of length over $\frac{\ell}{2}$ (otherwise, there won’t be any palindrome of length ℓ), each string has at most $\ell - 1$ prefix/suffix, total number of prefix/suffix is $O(N)$, by using subtraction, all Karp-Rabin fingerprints can be computed in $O(N)$ time. Step 1 and 2 take $O(N)$ time, step 3 takes $O(N \log N)$ time. Let $p = N^{d+2}$ in fingerprints, the probability of using brute force in step 5 is $O(\frac{1}{N^d})$, expected time complexity is $O(\ell N \times \frac{1}{N^d})$, which is smaller than $O(N)$ by setting $d = 1$. Total time complexity is expected to be $O(N \log N)$.